Lecture 2
Web Programming

HTTP Protocol and HTML

The Backbone of Web Communication and Content Structure

Mahhek Tahir

Overview of the HTTP Protocol

Definition: HTTP (HyperText Transfer Protocol) is the protocol used for transferring data over the web. It enables communication between web clients (browsers) and web servers.

Why It's Important: HTTP is the foundation of data exchange on the World Wide Web.

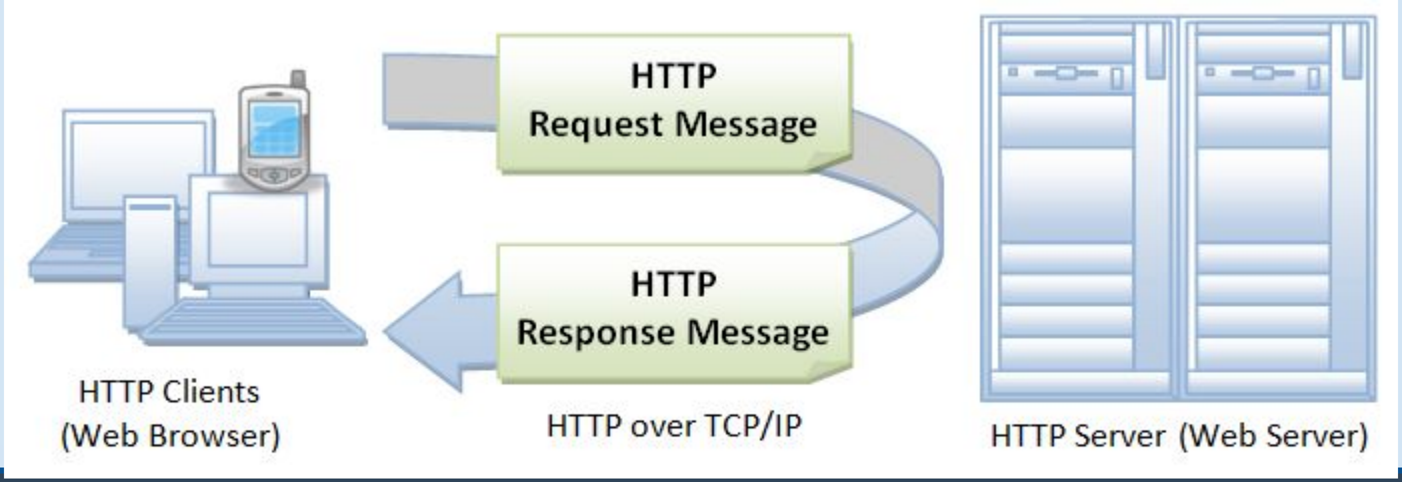
Key Point: HTTP is **stateless**, meaning each request is independent, and the server does not retain information about previous requests. Example.

How HTTP Works?

Request: A client (typically a web browser) sends an HTTP request to the web server. This request includes details like the type of request (GET, POST, etc.) and the resource (URL) being requested.

Processing: The server processes the request, retrieves the requested resource (HTML file, image, etc.), and prepares a response.

Response: The server sends an HTTP response back to the client, typically containing the requested resource or an error message if the resource cannot be found.



HTTP Request Structure

An HTTP request is made up of several parts:

- **Request Line**: Contains the HTTP method, the URL, and the HTTP version.
  - Example: `GET /index.html HTTP/1.1`
- **Headers**: Contain metadata about the request, such as the type of content requested or user-agent (browser).
  - Example: `User-Agent: Mozilla/5.0`
- **Body (optional)**: Contains data sent with the request (e.g., form submissions).

Example:

```
GET /home HTTP/1.1

Host: www.example.com
User-Agent: Mozilla/5.0
```

HTTP Response Structure

The HTTP response sent from the server contains:

1. **Status Line**: Includes the HTTP version, status code, and a brief description of the status.
   - Example: `HTTP/1.1 200 OK`
2. **Headers**: Provide metadata about the response, such as content type, server information, or cookies.
   - Example: `Content-Type: text/html`
3. **Body**: The content of the response (HTML, images, JSON, etc.).

Example:

less

Copy code

```
HTTP/1.1 200 OK

Content-Type: text/html

Content-Length: 1234
<html>...</html>
```

HTTP Status Codes

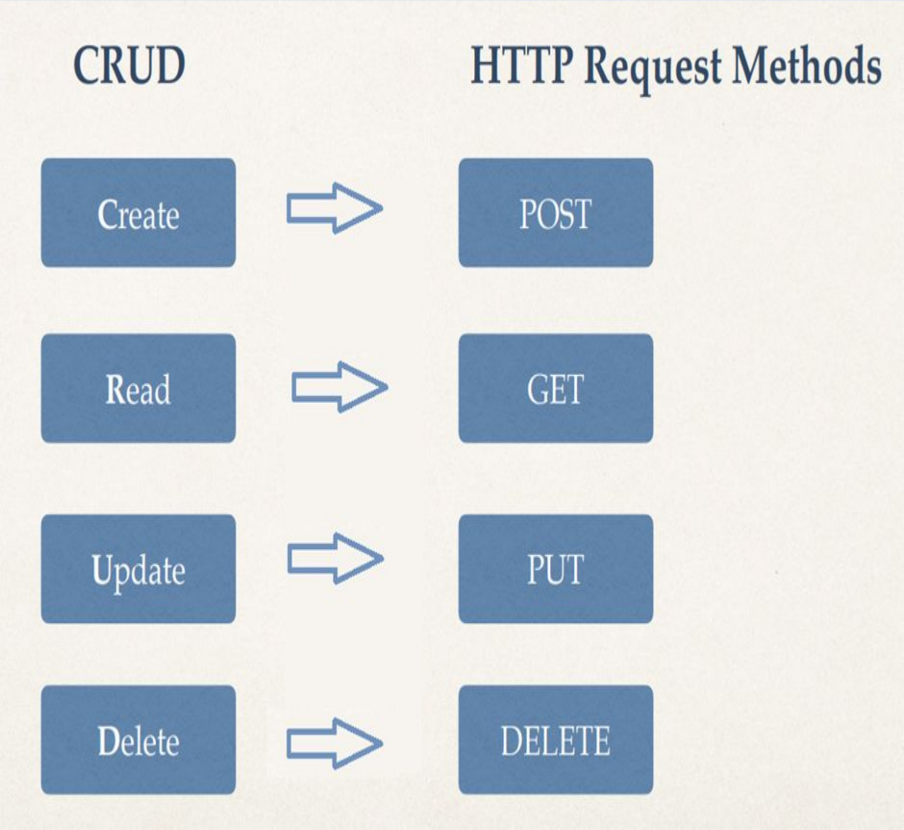
HTTP status codes indicate the result of a client's request. Some important status codes include:

- **200 OK**: The request was successful.
- **404 Not Found**: The requested resource could not be found on the server.
- **500 Internal Server Error**: There was an error on the server.
- **301 Moved Permanently**: The resource has been permanently moved to a new location.
- **403 Forbidden**: The server understands the request but refuses to authorize it.

HTTP Methods

HTTP defines several methods for interacting with resources:

1. **GET**: Retrieve a resource (e.g., an HTML file, image).
2. **POST**: Send data to the server (e.g., form submission).
3. **PUT**: Update a resource on the server.
4. **DELETE**: Remove a resource from the server.
5. **HEAD**: Retrieve only the headers (no body content).
6. **PATCH**: Apply partial modifications to a resource.



GET vs POST

- GET requests can be cached
- GET requests remain in the browser history
- GET requests can be bookmarked
- GET requests have length restrictions
- GET requests are to retrieve data

- POST requests are never cached
- POST requests do not remain in the browser history
- POST requests cannot be bookmarked
- POST requests have no restrictions on data length
- POST requests should be used to transmit data

GET requests should never be used when dealing with sensitive data

Request Response Cycle

- GET http://www.google.com HTTP/1.1

Host: www.google.com

Request

- HTTP/1.1 200

Content-type: text/html

Content-length: *xx*

```
<html>
    <head> … </head>
    <body> Google </body>
</html>
```

Response

Introduction to HTML

- **Definition**: HTML is the standard language used to create and design web pages.
- **Purpose**: It structures the content on a webpage, such as text, images, videos, and links.

Key Point: HTML uses **elements** (tags) to define the structure and content of a webpage.

HTML Elements Structures

Element comprises of:

- Start Tag
- End Tag
- Attributes and values
- Content

<tag attr1=”value1” attr2=”value2” ...> content…
</tag>

<tag attr1=”value1” attr2=”value2” … />

Basic HTML Structure

A basic HTML document includes several essential elements:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport"
content="width=device-width, initial-scale=1.0">

  <title>My Web Page</title>

</head>

<body>

  <h1>Welcome to My Web Page</h1>

  <p>This is a paragraph of text.</p>

</body>

</html>
```

- **`<!DOCTYPE html>`**: Declares the document type.
- **`<html>`**: Root element of the document.
- **`<head>`**: Contains metadata about the webpage (title, character encoding).

Key HTML Elements

Text Elements:

- `<h1>` **to** `<h6>`: Headings (from largest to smallest).
- `<p>`: Paragraph.
- `<a>`: Anchor (for links).
- `<ul>`: Unordered list.
- `<ol>`: Ordered list.
- `<li>`: List item.

Media Elements:

- `<img>`: Embeds an image.
- `<audio>`: Embeds an audio file.
- `<video>`: Embeds a video file.

Form Elements:

- `<form>`: Defines a form for user input.
- `<input>`: Defines an input field.
- `<textarea>`: Defines a text area.

Heading and paragraph

```
<!DOCTYPE html>
<html>
<body>
<h1>My First Heading</h1>
<h2>This is heading 2</h2>
<h3>This is heading 3</h3>
..
<h6>This is heading 6</h6>
<p>My first paragraph.</p>
</body>
</html>
```

Links and navigation - Anchor Tag

<!DOCTYPE html>

<html>

<body>

<h1>My First Heading</h1>

This is a link

</body>

</html>

Relative vs. Absolute URLs:

- **Relative URL**: Links to a resource within the same website (e.g., `page2.html`).
- **Absolute URL**: Links to a resource on any website (e.g., https://www.example.com).

NOTE: target=”_blank” = > opens a new window

HTML Documents – image

```
<!DOCTYPE html>
<html>
<body>
<h1>My First Heading</h1>
<img src="sunflower.jpg"  width="104" height="142">
</body>
</html>
```

Note: src also have relative and absolute url

HTML Document - Buttons

```
<!DOCTYPE html>
<html>
<body>
<h2>HTML Buttons</h2>
<p>HTML buttons are defined with the button tag:</p>
<button>Click me</button>
</body>
</html>
```

HTML Forms

Forms allow users to submit data to a server:

- **<form>**: Defines the start of a form.
- **<input>**: Defines an input field (e.g., text box, button).
- **<label>**: Defines a label for input elements.
- **<select>**: Defines a drop-down menu.
- **<textarea>**: Defines a multi-line text input.

Example:

```
<form action="/submit" method="POST">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name">
  <input type="submit" value="Submit">
</form>
```

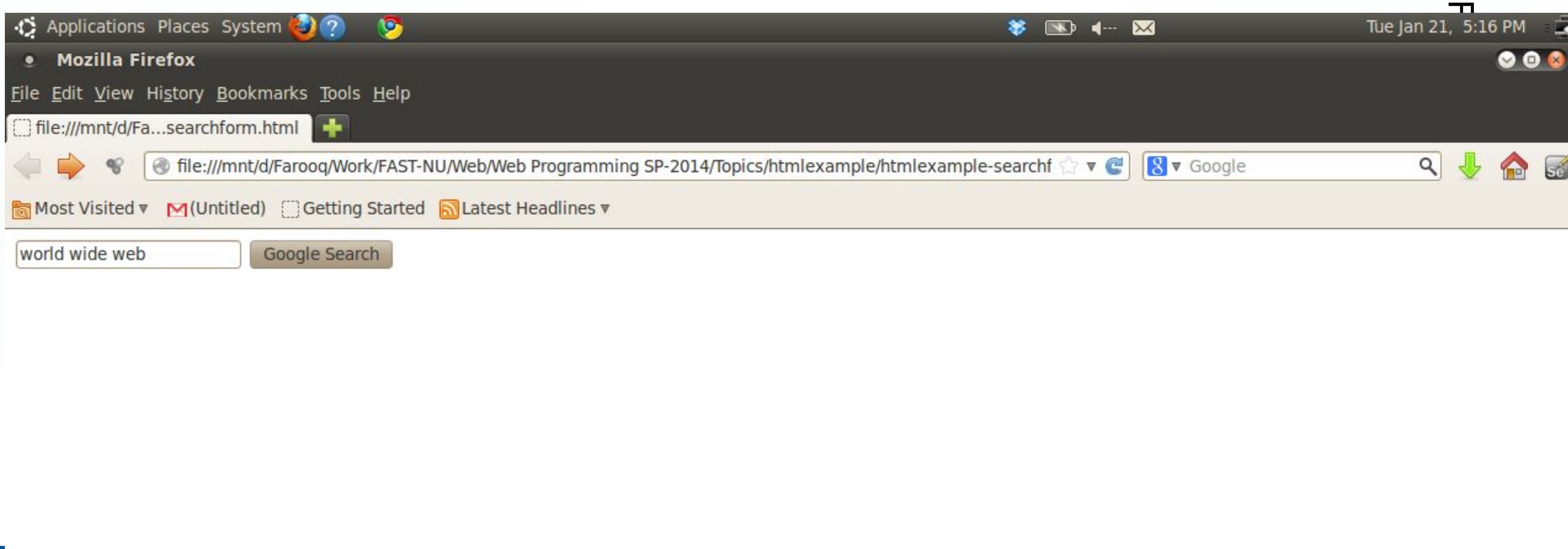
Request method

URL for form submission

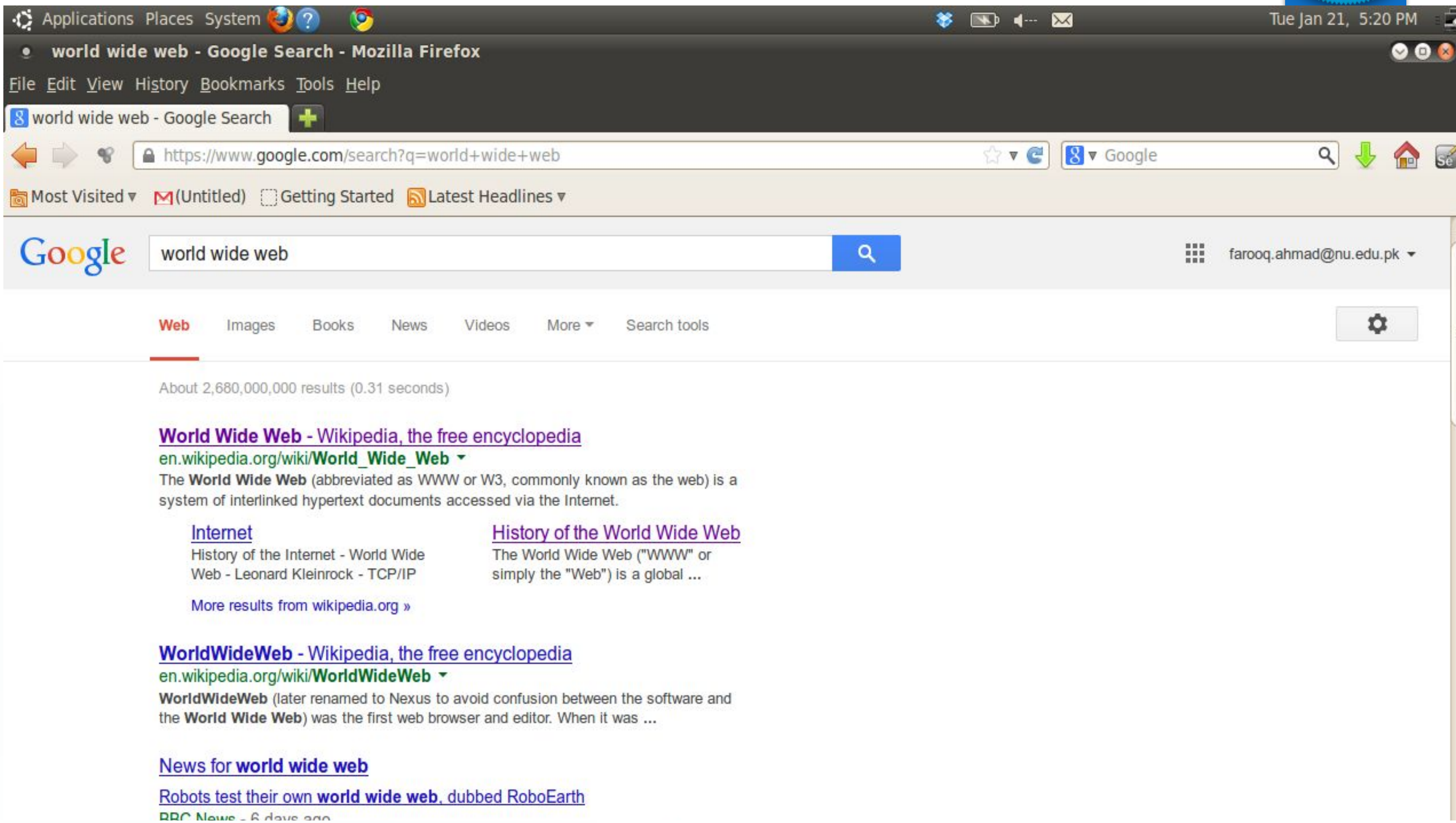
```
<html>
  <body>
       <form method="GET" action="http://www.google.com/search">
              <input type="text" name="q" />
              <input type="submit" value="Google Search" />
       </form>
  </body>
</html>
```

Text field

Represents a button that will submit form on click



Form parameters are shown in browser bar



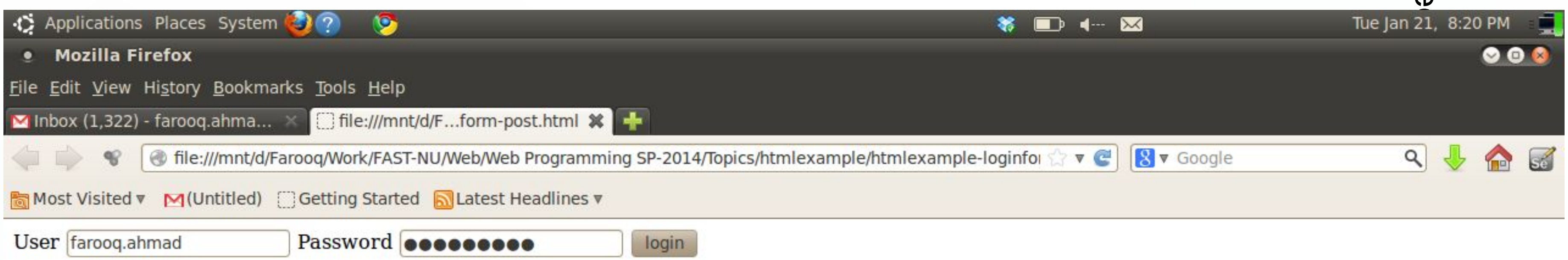
Request method

URL for form submission

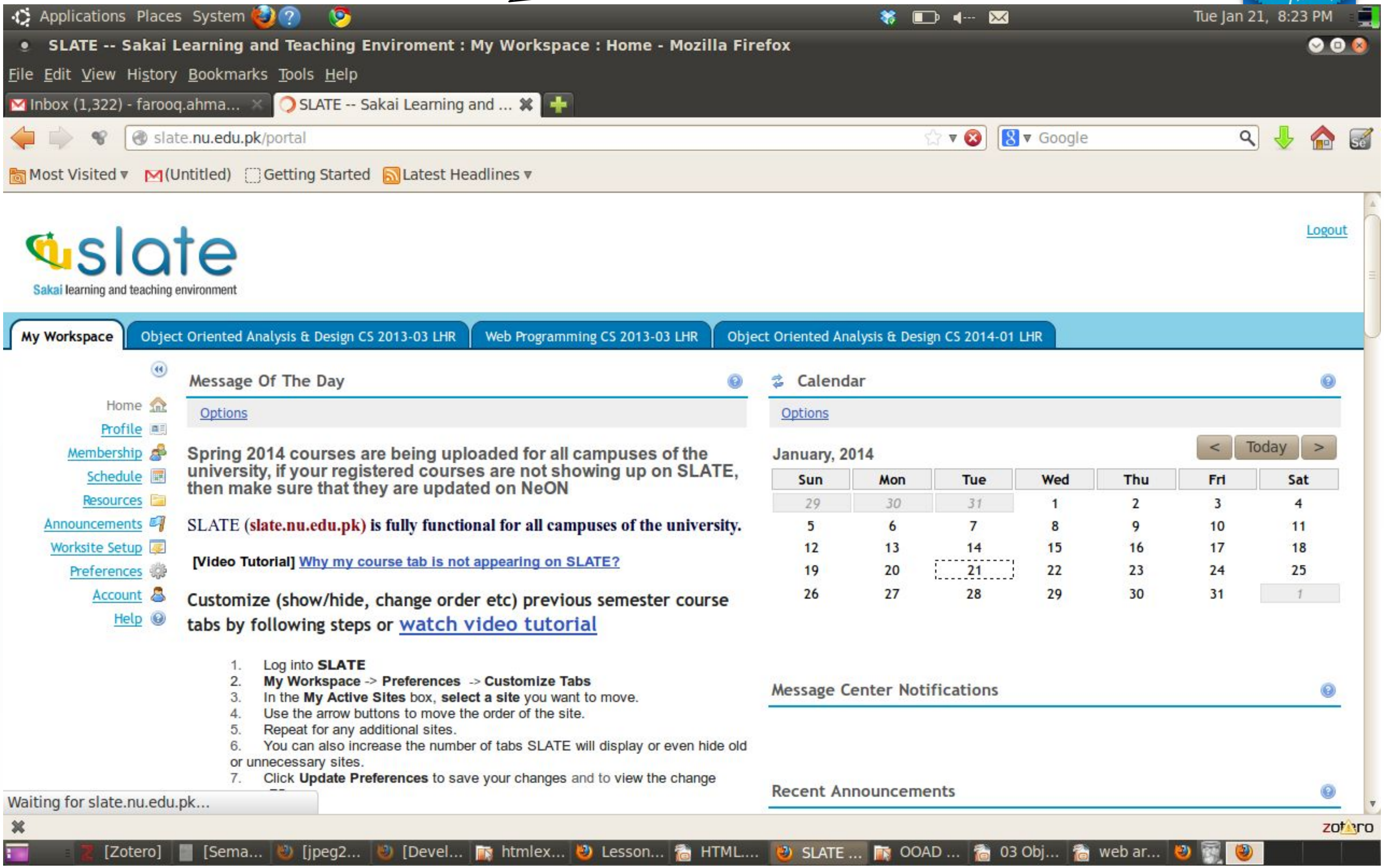
```
<html>
<body>
	<form method="POST" action="http://slate.nu.edu.pk/portal/xlogin">
		User <input type="text" name="eid" />
		Password <input type="password" name="pw" />
		<input type="submit" value="login">
	</form>
</body>
</html>
```

Text field

Password field



Form parameters not shown



Types of Input

- **1. Text**
- Allows the user to enter a single line of plain text.
- `<input type="text" placeholder="Enter name">`

- **2. Password**
- Hides the text.
- `<input type="password" placeholder="Password">`

- **3. File**
- Let’s the user upload a file. A Filepicker is displayed.
- `<input type="file" accept="image/*">`

Types of Input

- **1. Checkbox**
- Represents a binary choice (checked/unchecked).
- <input type="checkbox" id="subscribe">

- **2. Radio**
- Allows selecting one option from a group of options. Multiple radio buttons that form part of the same group will have the same name value.
- <input type="radio" name="gender" value="male">
<input type="radio" name="gender" value="female">
- **3. Date**
- Allows a user to select a date using a datepicker.
- <input type="date">.

HTML Tables

HTML tables are used to display data in a grid format.

- **<table>**: Defines the table.
- **<tr>**: Table row.
- **<td>**: Table data (cell).
- **<th>**: Table header.

```
<table>
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>John</td>
    <td>30</td>
  </tr>
</table>
```

Table

To make a cell span over multiple rows, use the `rowspan` attribute

To make a cell span over multiple columns, use the `colspan` attribute:

<table>
<tr><th colspan="2">NAME</th><th></th></tr>
<tr><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td></td></tr>
</table>

<table>
<tr><th rowspan="2">APRIL</th><td></td><td></td></tr>
<tr><td></td><td></td></tr>
<tr><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td></td></tr>
</table>

Solution

```
<table>
	<tr>
		<td colspan=2>Name</td>
		<td></td>
	</tr>
	<tr>
		<td></td>
		<td></td>
		<td></td>
	</tr>
	<tr>
		<td></td>
		<td></td>
		<td></td>
	</tr>
	<tr>
		<td></td>
		<td></td>
		<td></td>
	</tr>
</table>
```

```
<table>
	<tr>
		<td rowspan=3>April</td>
		<td></td>
		<td></td>
	</tr>
	<tr>
		<td></td>
		<td></td>
	</tr>
	<tr>
		<td></td>
		<td></td>
	</tr>
	<tr>
		<td></td>
		<td></td>
		<td></td>
	</tr>
</table>
```

Block Level Elements

These elements take up the full width of their parent container and start on a new line. They stack vertically and usually occupy the entire available horizontal space.

Examples:
<div>
<p>
<h1>, <h2>, <h3>, etc.

Inline Elements

These elements only take up as much width as their content. They do not start on a new line and only occupy the space they need, staying within the flow of the content around them.

Examples:
<span>
<label>
<img>

HTML Comment Tags

```
<!DOCTYPE html>
<html>
<body style="background-color : powderblue;">
<!-- Write your comments here -->
<h1 style="text-align:center;">Centered Heading</h1>
<p style="text-align:center;">Centered paragraph.</p>
</body>
</html>
```

HTML Document - list

- **Ordered List**

<ol>

*
Coffee
*

*
Tea
*

*
Milk
*

*
*

- **Unordered List**

<ul>

*
Coffee
*

*
Tea
*

*
Milk
*

*
*

HTML5 Features

HTML5 introduced new features for better web experiences:

1. **`<article>`**: Represents a self-contained piece of content.
2. **`<section>`**: Represents a section of a document.
3. **`<nav>`**: Defines navigation links.
4. **`<video>` and `<audio>`**: Embed media content without needing plugins.

Enclosing Tags

These tags require both opening and closing tags.

They are used for elements that enclose content and can contain child elements.

Format: <tagName>content</tagName>

```
<p>This is a paragraph.</p>
<div>
  <h1>Heading</h1>
  <p>Some text inside a div.</p>
</div>
<button>Click Me</button>
```

Self-closing Tags

These tags do not need a separate closing tag.

They are typically used for elements that cannot contain child elements.

Format: <tagName />

```
<img src="image.jpg" alt="An image" />
<input type="text" />
<br />
```

Class Test

- Create an unordered list containing the names of any three provinces of Pakistan. Then, create three nested ordered lists within the unordered list, containing names of any two cities of each province.